TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

METODOLOGÍA DE SISTEMAS II

Trabajo Práctico: Entrega Continua y Resiliencia del Software

Parte 1: Integración Continua (CI)

1. Filosofía de Trabajo: Explique la importancia de la filosofía "Fail-Fast" dentro de un Pipeline de CI. ¿Cómo beneficia esta práctica a la agilidad del equipo y al ahorro de recursos técnicos?

Es vital porque detiene el pipeline de inmediato ante la primera falla (ej. error de sintaxis), evitando gastar tiempo y recursos ejecutando pruebas complejas o costosas. Beneficia la agilidad al proporcionar feedback rápido y directo al desarrollador.

2. Anatomía del Pipeline (Stages): Para un proyecto profesional, describa qué validaciones se realizan en las siguientes etapas y qué tipo de riesgo mitiga cada una:

- Lint & Static Analysis: Verifica el formato, estilo y detecta patrones de código peligrosos. Mitiga el riesgo de introducir código desestructurado o con malas prácticas.
- Unit Tests: Ejecuta pruebas automáticas de la lógica individual de cada componente. Mitiga el riesgo de introducir fallos funcionales.
- Security Scan: Audita las dependencias en busca de vulnerabilidades de seguridad conocidas. Mitiga el riesgo de incorporar librerías vulnerables a producción.

3. Pipeline as Code: ¿Qué significa que el archivo de configuración (YAML) sea el "cerebro" del pipeline y por qué es una buena práctica que resida dentro del mismo repositorio de código?

Significa que las instrucciones de automatización se configuran mediante un archivo de texto estructurado. Es una buena práctica que resida en el mismo repositorio porque permite que el pipeline evolucione y se versione de forma conjunta y sincronizada con el código fuente.

Parte 2: Versionado Semántico (SemVer)

4. El Lenguaje del Cambio: Usted debe asignar un nuevo número de versión a un sistema que actualmente está en la v2.4.1. Indique a qué versión pasaría en los siguientes tres escenarios independientes:

- Escenario A: Se corrige un error en el cálculo de impuestos que no cambia la forma en que los usuarios usan el sistema.
  Pasa a la versión v2.4.2 (se incrementa el Patch).
- Escenario B: Se añade una nueva funcionalidad de "Exportar a PDF" que es totalmente compatible con las funciones actuales.
  Pasa a la versión v2.5.0 (se incrementa el Minor).
- Escenario C: Se elimina una función antigua y se cambia la forma de autenticarse, lo que requiere que otros sistemas que dependen del suyo realicen cambios para no romperse.
- Pasa a la versión v3.0.0 (se incrementa el Major).

5. Tags y Disparadores: Explique técnicamente cómo se vincula el uso de Tags en Git con la automatización del despliegue a producción.

Los números de versión SemVer se materializan en Git mediante *Tags*. Los pipelines de CD se configuran para "escuchar" la creación o envío de estos tags, utilizándolos como disparadores automáticos para iniciar el despliegue hacia producción.

Parte 3: Entrega y Despliegue Continuo (CD)

6. Entrega (Delivery) vs. Despliegue (Deployment): ¿Cuál es la diferencia crítica entre ambos conceptos en cuanto a la intervención humana? Mencione un caso de negocio donde sea preferible la "Entrega Continua" sobre el "Despliegue Continuo".

La diferencia crítica radica en la intervención humana: en la *Entrega Continua* el software está listo pero requiere aprobación manual para pasar a producción; en el Despliegue Continuo el proceso es 100% automático.

Caso de negocio: Es preferible la Entrega Continua en sistemas críticos o corporativos (como software bancario o médico) donde el negocio prefiere decidir el momento estratégico exacto del lanzamiento.

7. La Gestión de Entornos: Explique el flujo que sigue el código a través de los entornos de Dev, Staging y Prod. ¿Qué función específica cumple el entorno de Staging antes del paso final?

El código transita desde el entorno de desarrollo (Dev), pasa por pruebas rigurosas en Staging / QA, y finalmente se promueve a Producción (Prod).

Función de Staging: Actúa como réplica exacta de producción para realizar pruebas de aceptación de usuario (UAT), seguridad y estrés sobre el artefacto final antes del paso definitivo.

8. El Principio de Inmutabilidad: Defina qué es un Artefacto Inmutable y por qué es vital para la seguridad del sistema que no se vuelva a compilar el código cuando se mueve entre distintos entornos.

Un Artefacto Inmutable es un paquete compilado (binario o contenedor) que no se modifica una vez construido. Es vital para la seguridad porque garantiza que exactamente el mismo paquete validado en Staging sea el desplegado en Producción, evitando recompilaciones imprevistas o alteraciones de última hora.

Parte 4: Resiliencia y Mitigación de Riesgos

9. El Mecanismo de Rollback: Ante una falla crítica en el entorno real, explique:

- ¿En qué consiste un Rollback a nivel de infraestructura?
  Consiste en revertir el sistema a una versión estable anterior de manera casi instantánea al detectar anomalías en producción.
- ¿Cómo ayuda el haber tra1bajado con Tags de SemVer para que este proceso sea casi instantáneo?
  Permiten identificar con absoluta precisión el hito de la versión estable anterior para re-encender o redirigir el tráfico hacia su respectivo artefacto inmutable en segundos.

10. Estrategias Progresivas: Compare las estrategias Blue-Green Deployment y Canary Deployment.

- ¿En qué se diferencian respecto al manejo del tráfico de usuarios?
  Blue-Green mantiene dos entornos idénticos y conmuta el tráfico de golpe mediante un balanceador de carga; Canary libera la versión de manera gradual dirigiendo inicialmente un porcentaje menor de usuarios.
- ¿Cuál de las dos permite probar el código con un grupo reducido de usuarios reales antes del lanzamiento total?
  La estrategia que permite probar con un grupo reducido antes del lanzamiento total es Canary Deployment.